

2. Маніпулювання даними за допомогою Pandas

Pandas – це бібліотека Python, що використовується для аналізу даних. Дозволяє аналізувати великі дані, очищати невпорядковані набори даних, робити їх читабельними та релевантними (важливо в науці про дані).

Базові структури Pandas

Pandas зазвичай імпортують під `pd` псевдонімом¹. Створюється за допомогою ключового слова `as`.

```
import pandas as pd
import numpy as np
```

NumPy (Numerical Python) – це бібліотека, призначена для наукових обчислень, роботи з великими багатовимірними масивами та матрицями.

Основними базовими структурами в `pandas` є серії (`Series`) та датафрейми (`DataFrame`).

Серія – це одновимірний масив, що містить дані будь-якого типу (схоже на стовпець у таблиці).

Створити серію можна трьома способами: **зі списку** – програма присвоїть автоматичний індекс кожному елементу; **зі словника** – ключі словника автоматично стануть індексами серії, а значення її даними; **зі скаляра** – програма внесе одне значення на кілька позицій.

```
# зі списку
data = [10, 20, 30, 40]
s1 = pd.Series(data)

# зі словника
data_dict = {'Apple': 1.5, 'Banana': 0.5, 'Cherry': 3.0}
s2 = pd.Series(data_dict)

# зі скаляра
s3 = pd.Series(5, index=['a', 'b', 'c'])
```

¹ Псевдоніми (`alias`) – це альтернативна назва для того самого об'єкта.

DataFrame – це двовимірна структура даних, така як двовимірний масив або таблиця з рядками та стовпцями.

Створити датафрейм можна двома способами: **зі словника списків** – тут кожен ключ словника стає назвою стовпчика, а значення стають його даними; **зі списку словників** – тут кожен словник це один рядок, а його ключі це назви стовпчиків.

```
# зі словника списків
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'London', 'Paris']
}
df1 = pd.DataFrame(data)

# зі списку словників
data_rows = [
    {'Name': 'Alice', 'Age': 25},
    {'Name': 'Bob', 'Age': 30}
]
df2 = pd.DataFrame(data_rows)
```

Робота з реальними файлами

Простий спосіб зберігання великих наборів даних – це використання CSV-файлів, дослівно файлів, розділених комами (comma separated files).

Тут ми викликаємо функцію `read_csv`, призначену для читання даних з CSV-файлів та перетворення їх у структуру даних DataFrame. Якщо ж система не знайшла вказаного файлу, в консоль виведеться «cars.csv not found».

```
try:
    cars_df = pd.read_csv('cars.csv')
except FileNotFoundError:
    print("cars.csv not found")
```

Далі використовуються три самі основні методи бібліотеки: **head()** – огляд перших рядків; **info()** – інформація про датафрейм; **describe()** – обчислює основні статистичні показники таблиці.

```
display(cars_df.head(5))
cars_df.info()
display(cars_df.describe())
```

Внесені зміни зберігаються лише в оперативній пам'яті. Щоб вони не зникли після закриття програми, використовують методи `to_csv/to_parquet`². Вони беруть структуру датафрейму з пам'яті та записують її у файл на диску.

```
cars_df.to_csv('processed_cars.csv', index=False)
try:
    cars_df.to_parquet('processed_cars.parquet', index=False)
except ImportError:
    pass
```

Параметр `Index=False` каже програмі не зберігати технічні номери рядків. Інакше при повторному відкритті файлу з'явиться зайвий стовпчик `Unnamed: 0`.

Дослідження даних та маніпуляція над ними

Для того, щоб швидко отримати базову інформацію про таблицю та її структуру, використовують методи: `columns.tolist()` - перетворює назви стовпців датафрейму (об'єкти `Index`) у звичайний список; `index` - дозволяє побачити індекси таблиці; `shape` - виводить розмір таблиці (кількість рядків та стовпців).

```
print(f"Columns: {cars_df.columns.tolist()}")
print(f"Index: {cars_df.index}")
print(f"\nShape: {cars_df.shape}")
```

Для впорядкування даних використовують: `sort_values()` - сортує таблицю за значенням певного стовпця; `sort_index()` - сортує за індексами.

```
display(cars_df.sort_values(by='(kW)', ascending=False).head())
display(cars_df.sort_index().head())
```

Параметр `ascending=False` означає сортування за спаданням. Також сортування може містити параметр `inplace=False` (за замовчуванням) - повертає нову змінену копію датафрейму, не змінюючи оригінальні дані.

Зміна конкретних даних у таблиці виконується через метод `loc` - дозволяє звертатися до елементів за індексом рядка та назвою стовпця.

```
cars_df.loc[0, 'YEAR'] = 2025
cars_df.loc[0, 'YEAR'] = 2012
```

² Формат `Parquet` є бінарним, важить у кілька разів менше звичайного `CSV` та широко використовується у `Big Data`.

Для видалення даних використовується метод `drop()` – він дозволяє видаляти як стовпці так і рядки. За замовчуванням повертає новий датафрейм, тому результат потрібно зберегти у змінну.

```
temp_df = cars_df.drop(columns=['RATING', 'Unnamed: 5'])
temp_df = temp_df.drop(index=0)
```

Доступ до даних та їх індексація

Для вибору стовпців використовують квадратні дужки. Якщо вказати одну назву – повернеться серія, якщо список назв – повернеться датафрейм (підтаблиця).

```
makes = cars_df['Make']
subset = cars_df[['YEAR', 'Make', 'Model']]
```

Для доступу до рядків використовують два основні методи: `iloc` – працює з позицією рядку в таблиці (`iloc[0]` повертає перший рядок, незалежно від індексу); `loc` – працює з індексами або назвами (`loc[6]` повертає рядок з індексом 6).

```
print(cars_df.iloc[0])
print(cars_df.iloc[4])
print(cars_df.loc[6])
print(cars_df.loc[5, 'Model'])
```

Індекси можна змінювати вручну. Наприклад, інколи потрібно перемішати дані (`shuffle`), щоб порядок рядків не впливав на аналіз або навчання моделі.

```
np.random.seed(42)
indexes = np.arange(cars_df.shape[0])
np.random.shuffle(indexes)
cars_df.index = indexes
cars_df.sort_index(inplace=True)
```

Для складніших перевірок використовують методи: `isin()` – перевіряє, чи входить значення до списку; `str.contains()` – для пошуку тексту всередині рядків.

```
cars_df[cars_df['Make'].isin(['BMW', 'Audi'])]
cars_df[cars_df['Model'].str.contains('X')]
```

Фільтрація та вибірка даних

Фільтрація та вибірка даних дозволяє працювати тільки з потрібною частиною таблиці.

Найпростіший варіант - за однією умовою. У квадратних дужках записується логічний вираз, який повертає True або False для кожного рядка. У випадку нижче залишаться тільки ті рядки, де значення у стовпці Make рівні Tesla.

```
tesla_cars = cars_df[cars_df['Make'] == 'TESLA']
```

Можна комбінувати кілька умов, використовуючи оператори: **&** - логічне «І»; **|** - логічне «Або»; **~** - логічне «Не». Кожну умову потрібно брати в окремі дужки.

```
tesla_2016 = cars_df[(cars_df['Make'] == 'TESLA') & (cars_df['YEAR'] == 2016)]
```

Для складніших випадків умови часто зберігаються в змінні.

```
mask = (cars_df['Make'] == 'FORD') & (cars_df['YEAR'] == 2014)
mask2 = (cars_df['Make'] == 'CHEVROLET') | (cars_df['YEAR'] == 2016)
```

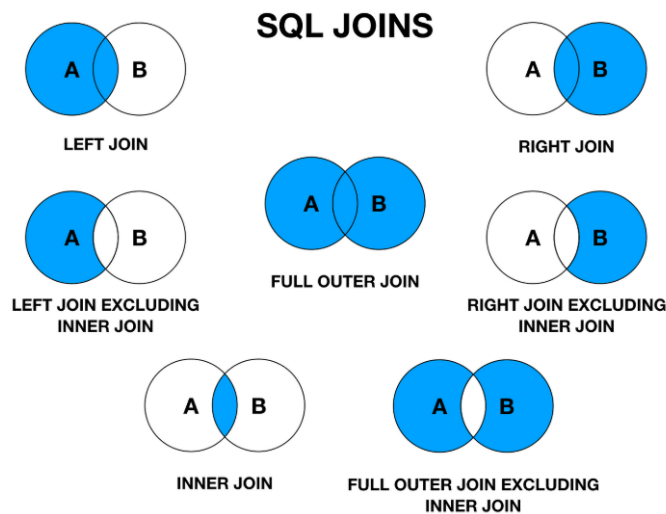
Для фільтрації за кількома можливими значеннями використовують метод `isin()`. У випадку нижче код поверне всі автомобілі марок BMW та KIA.

```
brands = ['BMW', 'KIA']
display(cars_df[cars_df['Make'].isin(brands)].head())
```

Для роботи з текстом застосовується метод `str.contains()`, який дозволяє знайти підрядки всередині тексту. Параметр `case=False` означає, що регістр не має значення. Код нижче знаходить всі моделі, в назві яких зустрічається слово battery.

```
display(cars_df[cars_df['Model'].str.contains('battery',
case=False)].head())
```

Об'єднання та агрегація даних



У роботі з реальними даними часто виникає потреба об'єднувати кілька таблиць або узагальнювати інформацію. Виділяють кілька основних типів об'єднань: **Inner join** – повертає тільки спільні елементи двох таблиць; **Left join** – повертає всі дані з лівої таблиці і лише відповіді з правої; **Right join** – навпаки, всі дані з правої таблиці; **Outer join** – об'єднує всі можливі значення з обох таблиць.

Конкатенація `pd.concat` використовується коли таблиці мають схожу структуру і їх потрібно з'єднати.

```
df_a = pd.DataFrame({'ID': [1, 2], 'Val': ['A1', 'A2']})
df_b = pd.DataFrame({'ID': [3, 4], 'Val': ['B1', 'B2']})

pd.concat([df_a, df_b], axis=0) #Вертикально – додає рядки один під одним
pd.concat([df_a, df_b], axis=1) # Горизонтально – додає стовпці поруч
```

Об'єднання `pd.merge` використовується коли потрібно з'єднати таблиці за спільним ключем (полем id або назвою).

```
left = pd.DataFrame({'key': ['K0', 'K1', 'K1', 'K3'], 'A': ['A0', 'A1', 'A2', 'A3']})
right = pd.DataFrame({'key': ['K0', 'K1', 'K1', 'K2'], 'B': ['B0', 'B1', 'B2', 'B3']})

pd.merge(left, right, on='key', how='inner')
```

Параметр `how="inner"` означає, що залишаться тільки ті рядки, де ключ є в обох таблицях.

Метод `groupby` розділяє дані на групи, після чого до кожної групи застосовується функція і результати об'єднуються назад. В прикладі нижче застосовуються функції: **mean()** – середнє значення; **max()** – максимальне; **count()** – кількість записів.

```
avg_kw = cars_df.groupby('Make')['km'].mean()
multi_agg = cars_df.groupby('Make')['km'].agg(['mean', 'max', 'count'])
```

Метод `pivot_table` дозволяє створювати зведені таблиці. В прикладі нижче наведені: `values` - що аналізуємо (пробіг); `index` - рядки (марка авто); `columns` - стовпці (рік); `aggfunc="mean"` - середнє значення.

```
pd.options.display.float_format = '{:.2f}'.format

pivot = cars_df.pivot_table(values='(km)', index='Make', columns='YEAR',
aggfunc='mean')
```

Очищення та аналіз даних

Реальні дані майже завжди містять помилки: пропущені значення, дублікати або некоректні записи. Тому перед аналізом їх потрібно почистити.

Щоб швидко знайти дірки в даних, використовують метод `isna().sum()`. Він показує кількість пропущених значень у кожному стовпці.

```
print(cars_df.isna().sum())
```

Після виявлення пропусків є два основні підходи їх виправлення: `dropna()` - видаляє всі рядки з пропущеними значеннями; `fillna()` - заповнює їх певними значеннями.

```
clean_df = cars_df.dropna()
filled_df = cars_df.fillna(999)
```

Інколи в таблиці можуть бути повторювані рядки (дублікати). Для їх пошуку використовують `duplicated()`, а для видалення `drop_duplicates()`.

```
students = pd.DataFrame({
    'student_id': [101, 102, 101],
    'name': ['Alice', 'Bob', 'Alice'],
    'score': [85, 92, 85]
})

students.duplicated().sum()
students.drop_duplicates()
```

Метод `duplicated().sum()` показує кількість повторів у таблиці.

Існують і більш розумні способи заповнення порожніх значень: `interpolate()` - обчислює проміжні значення на основі сусідніх; `ffill()` - підставляє попереднє відоме значення.

```
series_nan = pd.Series([1, np.nan, np.nan, 4])  
  
series_nan.fillna(0)  
series_nan.interpolate()  
series_nan.ffill()
```


2.1 Візуалізація даних

У pandas небажано використовувати цикли `for`, оскільки вони працюють повільно. Замість цього застосовується векторизація – виконання операцій одразу над усім стовпцем.

```
# Створюється DataFrame з 1 мільйоном чисел
large_df = pd.DataFrame({'nums': range(1_000_000)})

large_df['nums'].apply(lambda x: x + 10)  # Цикл
large_df['nums'] + 10                    # Векторизація
```

Векторизовані операції працюють значно швидше оскільки використовують оптимізації на рівні бібліотек (NumPy).

Method chaining – це стиль написання коду, коли операції виконуються ланцюжком. Такий код виглядає як послідовність дій і легше читається.

В коді нижче послідовно виконується: фільтрація – створення нового стовпця – сортування – вибір колонок – обмеження результату.

```
clean_chain = (
    cars_df
    .query("Make == 'TESLA'")
    .assign(power_per_km = lambda df: df['(kW)'] / df['(km)'])
    .sort_values(by='power_per_km', ascending=False)
    [['Model', 'power_per_km']]
    .head(3)
)
```

Іноді дані містять широкий формат – коли значення змінної (роки) записані в назвах стовпців. Для аналізу зручніший довгий формат, де ці значення винесені в окремий стовпець. Метод `melt()` перетворює таблицю у такий вигляд.

```
df_wide = pd.DataFrame({'City': ['London'], '2022': [15], '2023': [18]})
df_long = df_wide.melt(id_vars='City', var_name='Year',
value_name='Temp')
```

У результаті роки перетворюються на значення одного стовпця, а відповідні їм дані (температура) на значення другого стовпця.

Для обробки дат використовується `pd.to_datetime` – перетворює аргументи (рядки, стовпці) у формат дати та часу. Після можна застосувати `.dt` для отримання окремих компонентів (день, місяць, день тижня тощо).

```
df_dates = pd.DataFrame({'date': ['2024-01-01']})
df_dates['date'] = pd.to_datetime(df_dates['date'])
df_dates['day'] = df_dates['date'].dt.day_name()
```

Pandas дозволяє візуально підсвічувати значення прямо в таблиця. Наприклад, `background_gradient` створює ефект `heatmap`³. Це допомагає швидко побачити великі та малі значення.

```
numeric_df = cars_df[['(kW)', '(km)', 'TIME (h)']]
numeric_df.head().style.background_gradient(cmap='YlGn')
```

Для швидкої візуалізації можна використовувати вбудований метод `.plot()`.

```
cars_df['(kW)'].plot(title='Power')
cars_df.plot.scatter(x='(kW)', y='(km)')
```

Задати тип діаграми можна також за допомогою аргументу `kind`.

```
cars_df.plot(kind='scatter', x='(kW)', y='(km)')
```

Основні типи графіків у Pandas

line	лінійний графік, ідеальний для часових рядів
bar / barh	вертикальна / горизонтальна стовпчикова діаграма для порівняння категорій
hist	гістограма для відображення розподілу даних
box	ящик з вусами для візуалізації описової статистики (медіана)
area	графік площі (зафарбований лінійний графік)
scatter	діаграма розсіювання для відображення залежності між двома числовими змінними
pie	Кругова діаграма

³ Ефект `heatmap` (теплова карта) – це спосіб візуалізації даних, при якому інтенсивність певного показника відображається різними кольорами.